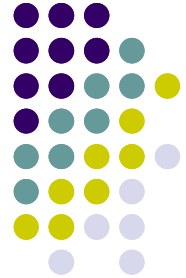


Design Patterns In Real-Time Game

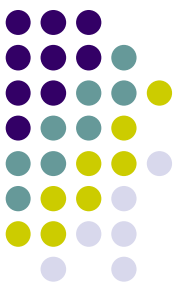


Architecture of Real-Time Systems
SE 456

Ed Keenan

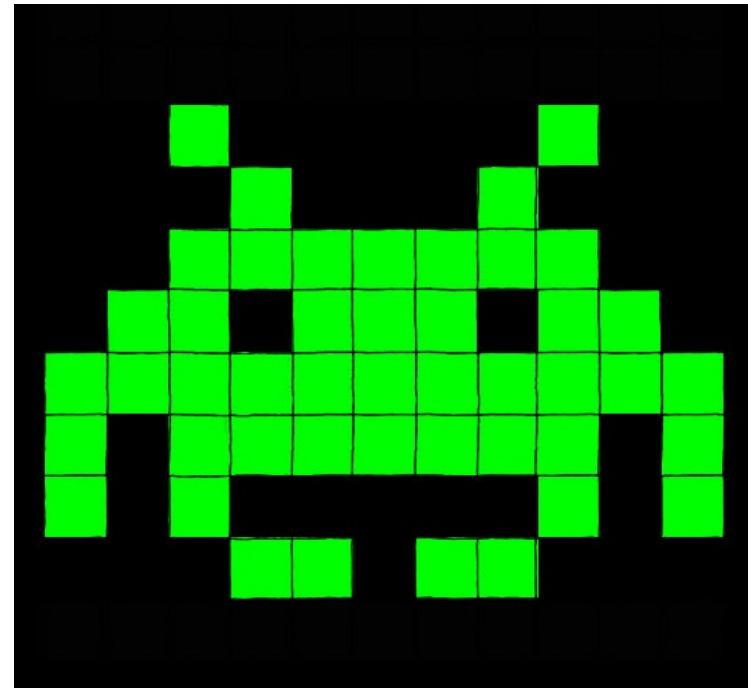
10 March 2021

13.0.8.6.1 Mayan Long Count



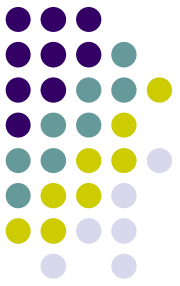
Overview

- Study of several problems
 - Using Design Patterns

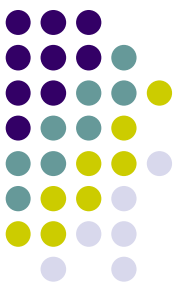


Problem 1

Creation



- Need a clean way to create objects
 - Aliens
 - Different types of Aliens
 - Crabs
 - Octopus
 - Squid
 - Different Locations
 - Grid of 5 x 11 Aliens
 - 55 aliens all together
 - Shields
 - Need 4 of them
 - Same as the others just different position



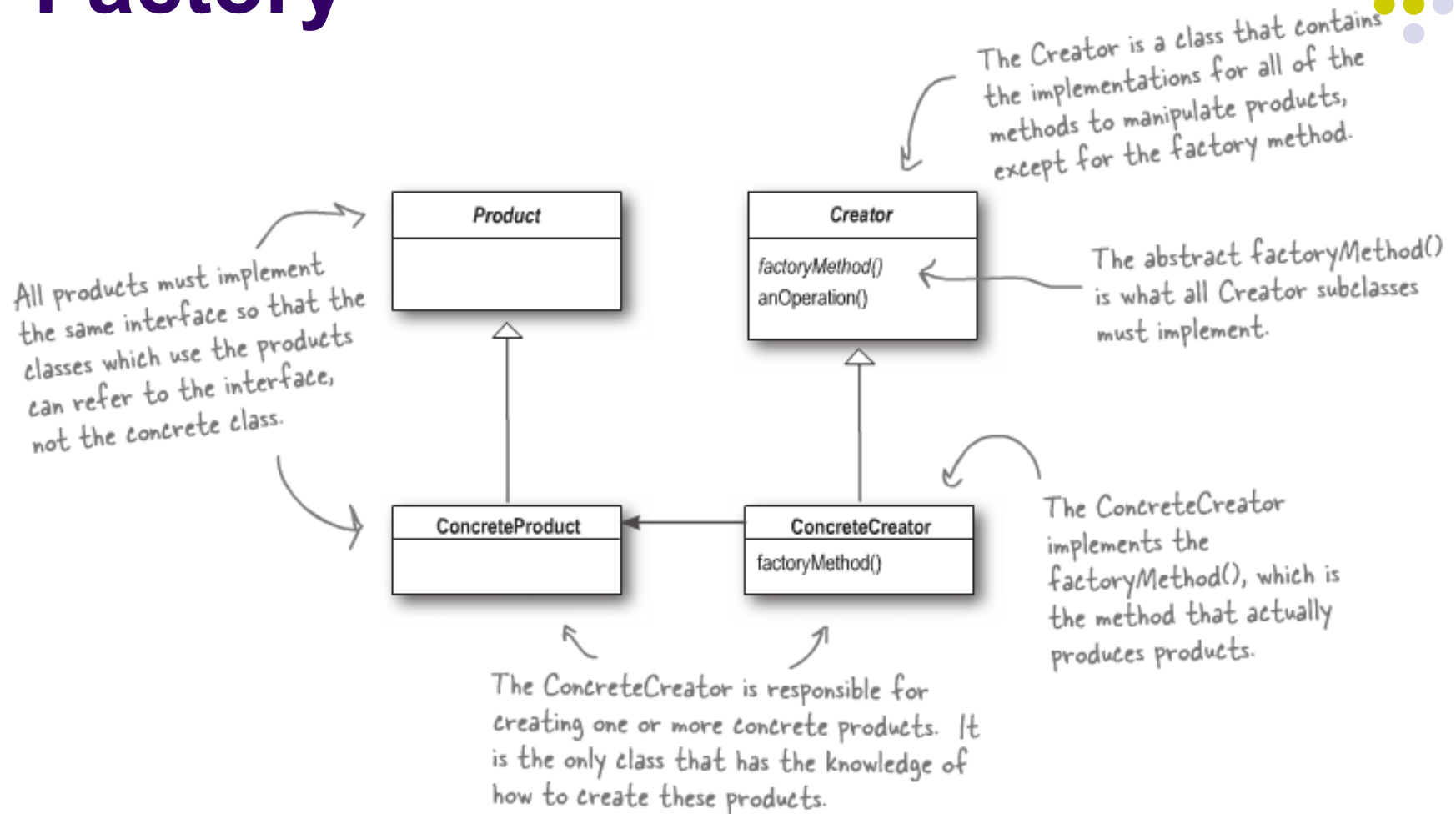
Problem 1

Repeated Creation

- Work this issue backwards
 - Where's that Elf...
- How would you create a Alien?
 - You know the type (say Crab)
 - You know the location (3 row, 5 col)
 - You have a name (Crab)
- Factory?
 - AlienFactory.Create(**Crab**, **Row_3**, **Col_5**);

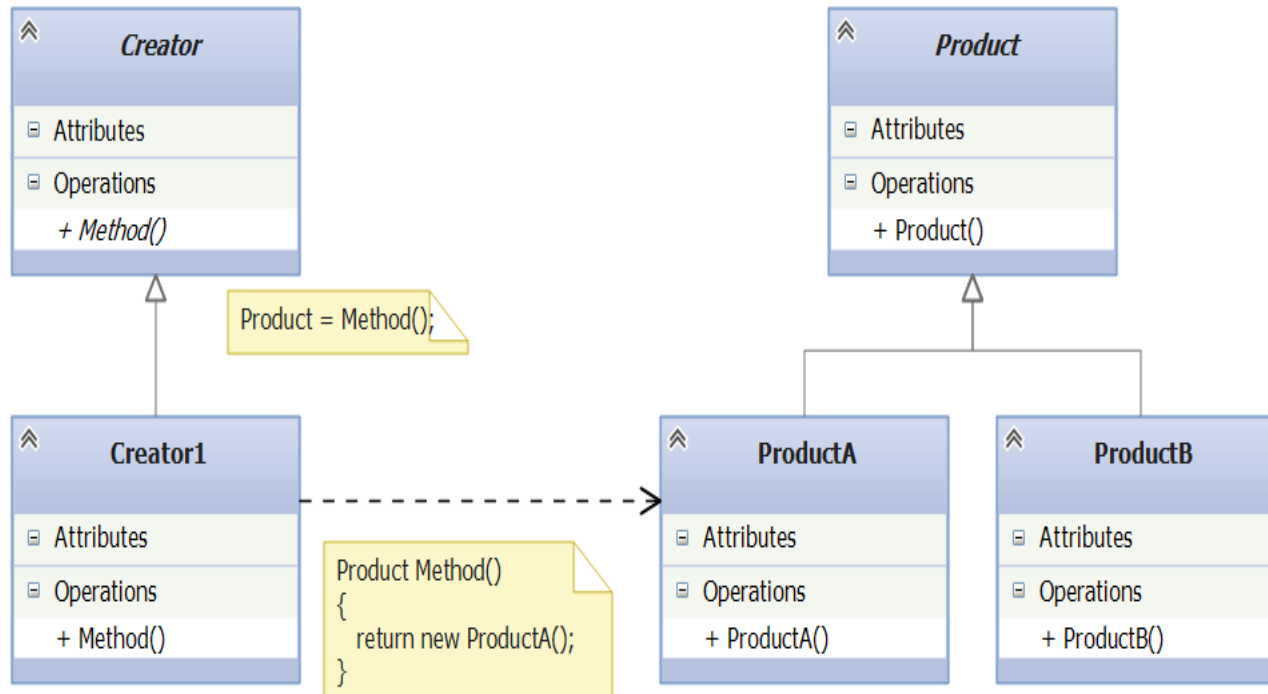
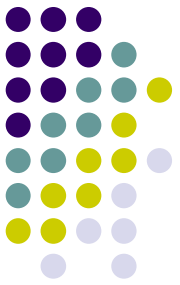
Problem 1

Factory



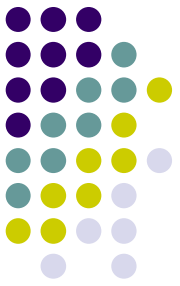
Problem 1

Responsibilities

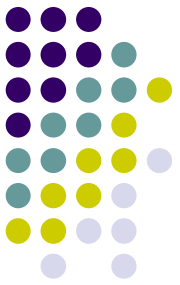


Problem 2

Organization



- Organization
 - Who holds the all my data?
 - Where does it live
 - Where do I store the references/names?
 - 100s of items to control
- How is it associated to the game layout?
 - Game()
 - Initialize()
 - LoadContent()
 - Update()
 - Draw()



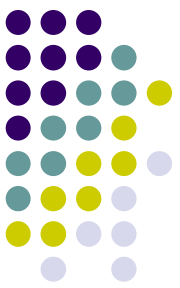
Problem 2

Access to managers

- You should know this one...
 - Managers
 - Hold data
 - Organize data
 - No hardcode data elements
 - Why Singletons?
 - Easy access anywhere.
 - One and only one

Problem 2

Singleton



The `getInstance()` method is static, which means it's a class method, so you can conveniently access this method from anywhere in your code using `Singleton.getInstance()`. That's just as easy as accessing a global variable, but we get benefits like lazy instantiation from the Singleton.

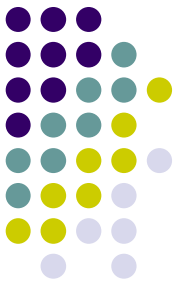
Singleton
static <code>uniqueInstance</code>
// Other useful Singleton data...
static <code>getInstance()</code>
// Other useful Singleton methods...

The `uniqueInstance` class variable holds our one and only instance of Singleton.

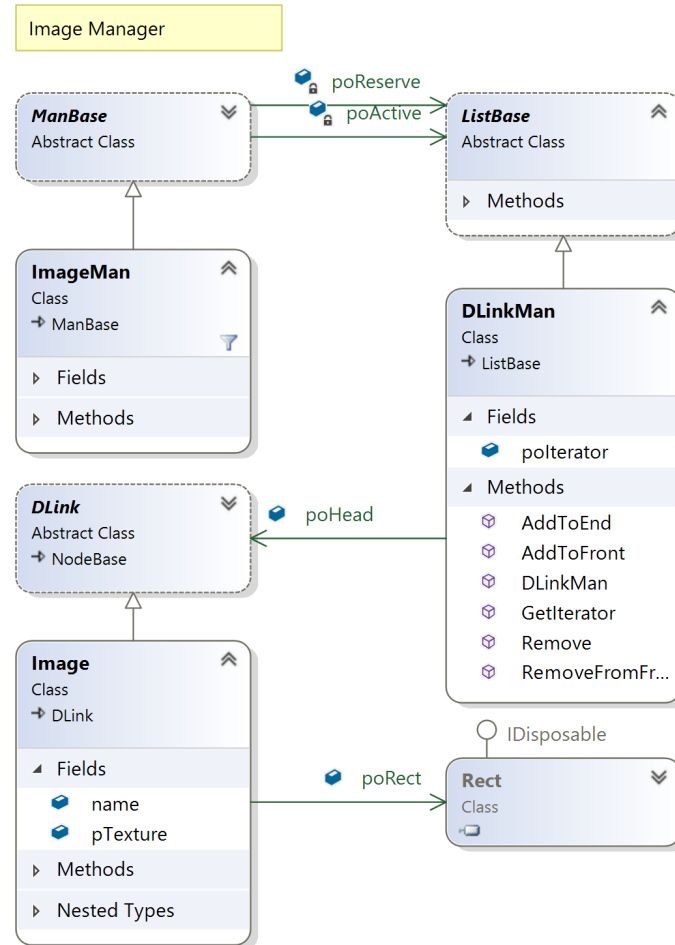
A class implementing the Singleton Pattern is more than a Singleton; it is a general purpose class with its own set of data and methods.

Problem 2

Manager

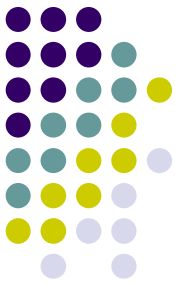


- Reusable manager
 - Same pattern repeating
 - Abstracted
- Basic API
 - Add
 - Create
 - Find
 - Remove
- Singleton
 - Easy global access
- Enums instead of String
 - Faster
 - No typos

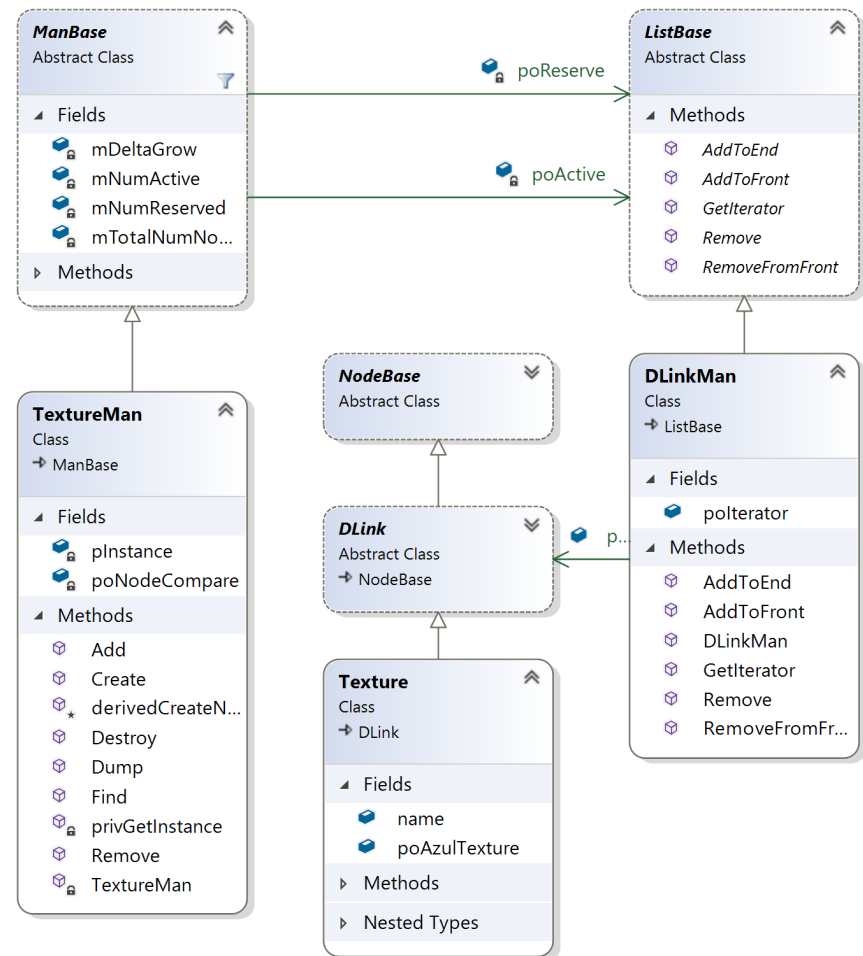


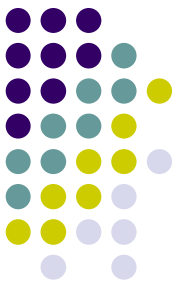
Problem 2

Object Pool



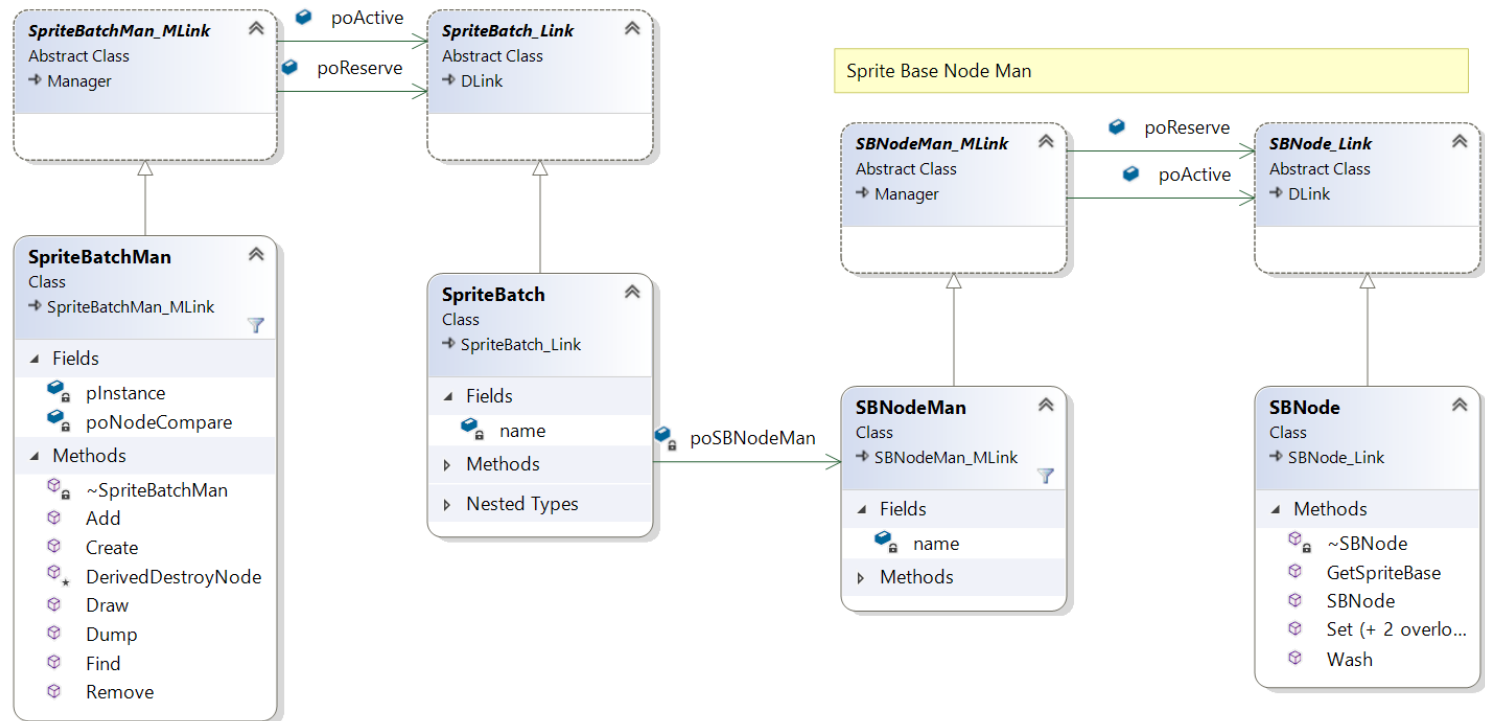
- **Object Pools**
 - Don't recreate (new) many same objects... reuse the in a clean way.
 - Can Add(), Remove(), and Find() objects





Problem 2

Sprite Batch Man

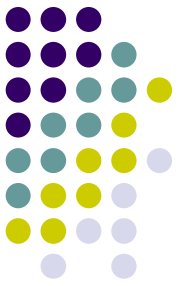


- **Less cost**

- Don't want to pay the cost of creation and destruction because they are all initialized in the same state.
- SpriteBatchNode - is used and recycled, they are in pools

Problem 3

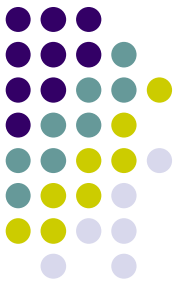
Reuse



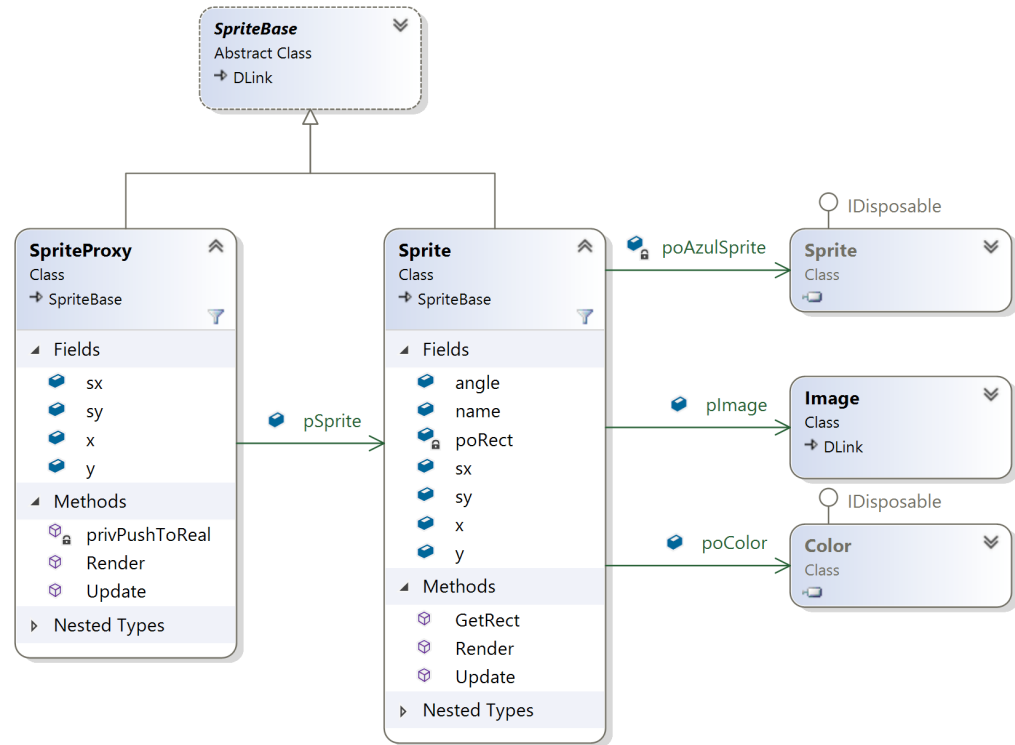
- How do we reuse a Animated Sprite?
 - Want to have animated sprites
 - Crab
 - Squid
 - Octopus
 - Reuse them in the Alien Grid
 - Setup once
 - Automated process
 - Unique instance data
 - Position

Problem 3

Proxy Sprites

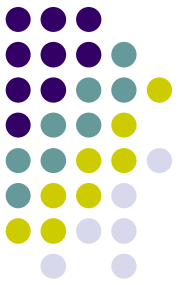


- **Proxy** objects
 - Reuse the existing object
 - Key point
 - Only one Animated Sprite
 - Instance data that is different
 - Position
 - Some small amount of data
 - Proxy points to Full object
 - Overwrite the differing data

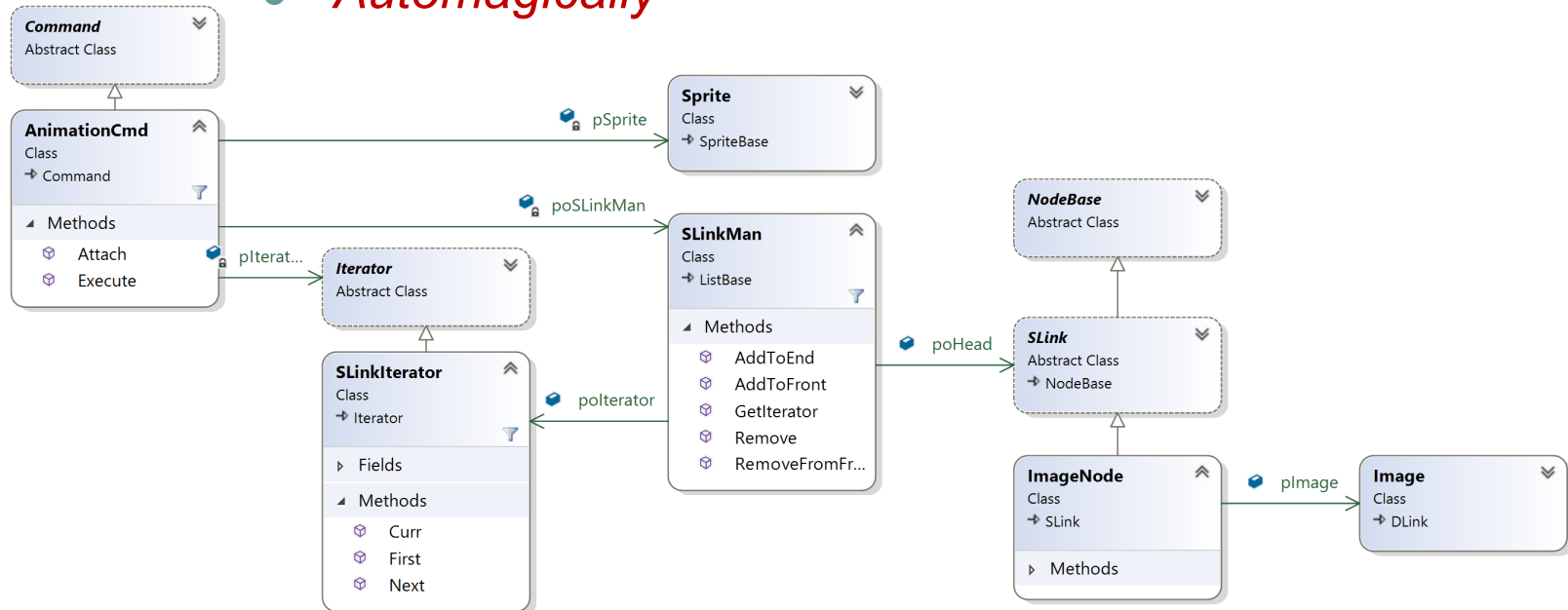


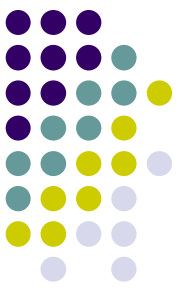
Problem 3

Animated Sprites



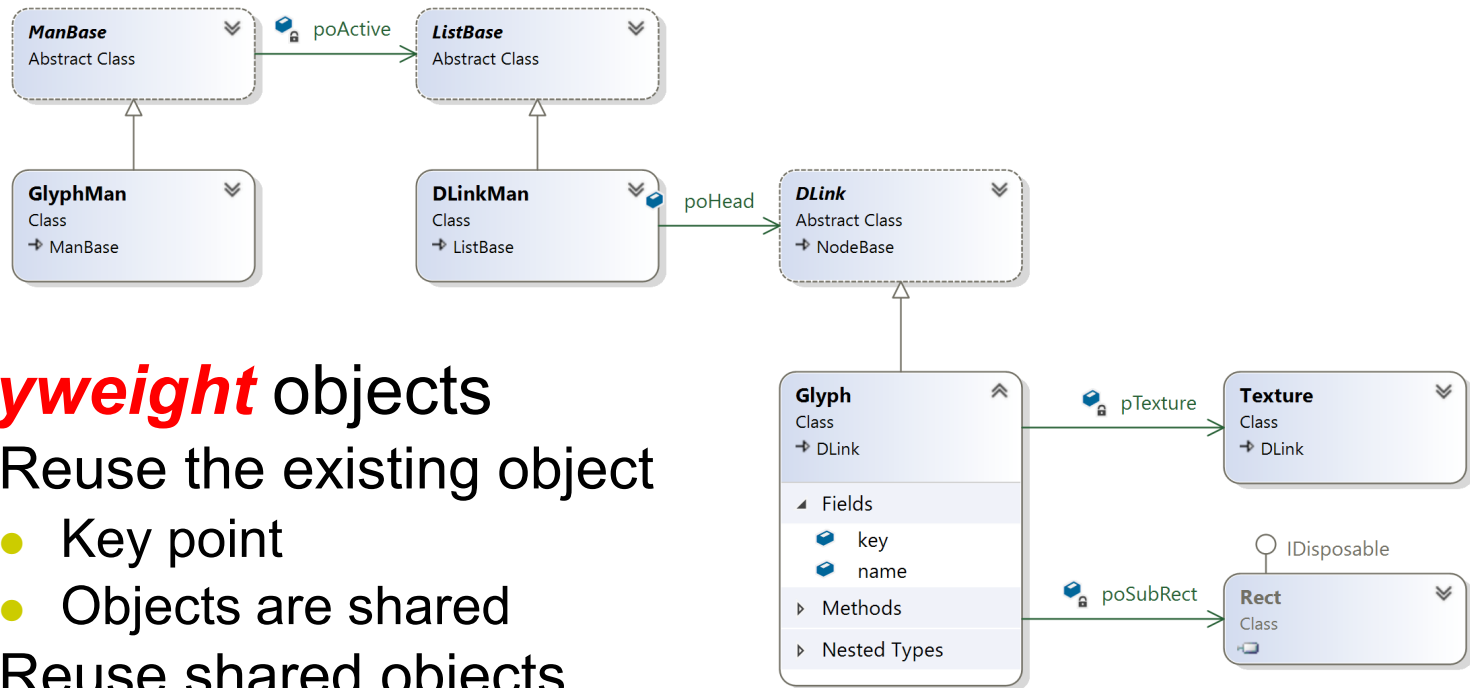
- Can have someone else animate the Sprite
 - Animation Sprite is flipping the Images
 - Delegation
- Now Our sprite benefits
 - *Automagically*





Problem 3

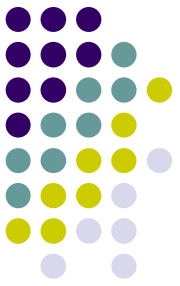
Flyweight Sprites



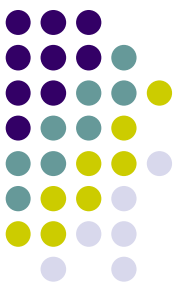
- **Flyweight** objects
 - Reuse the existing object
 - Key point
 - Objects are shared
 - Reuse shared objects
- Font system
 - Texture, Image
 - Same
 - Sets x,y renders

Problem 4

Hierarchy



- How do I organize data in a Hierarchy
 - Allow data / behavior / collisions interact in a Coarse to Fine approach
- Move data into structural relationships
 - Relationships implies rules
 - Shield system
 - Movement – Aliens as a group
 - Relationships optimize performance
 - Reduce work
 - Alien Grid – Efficient detection

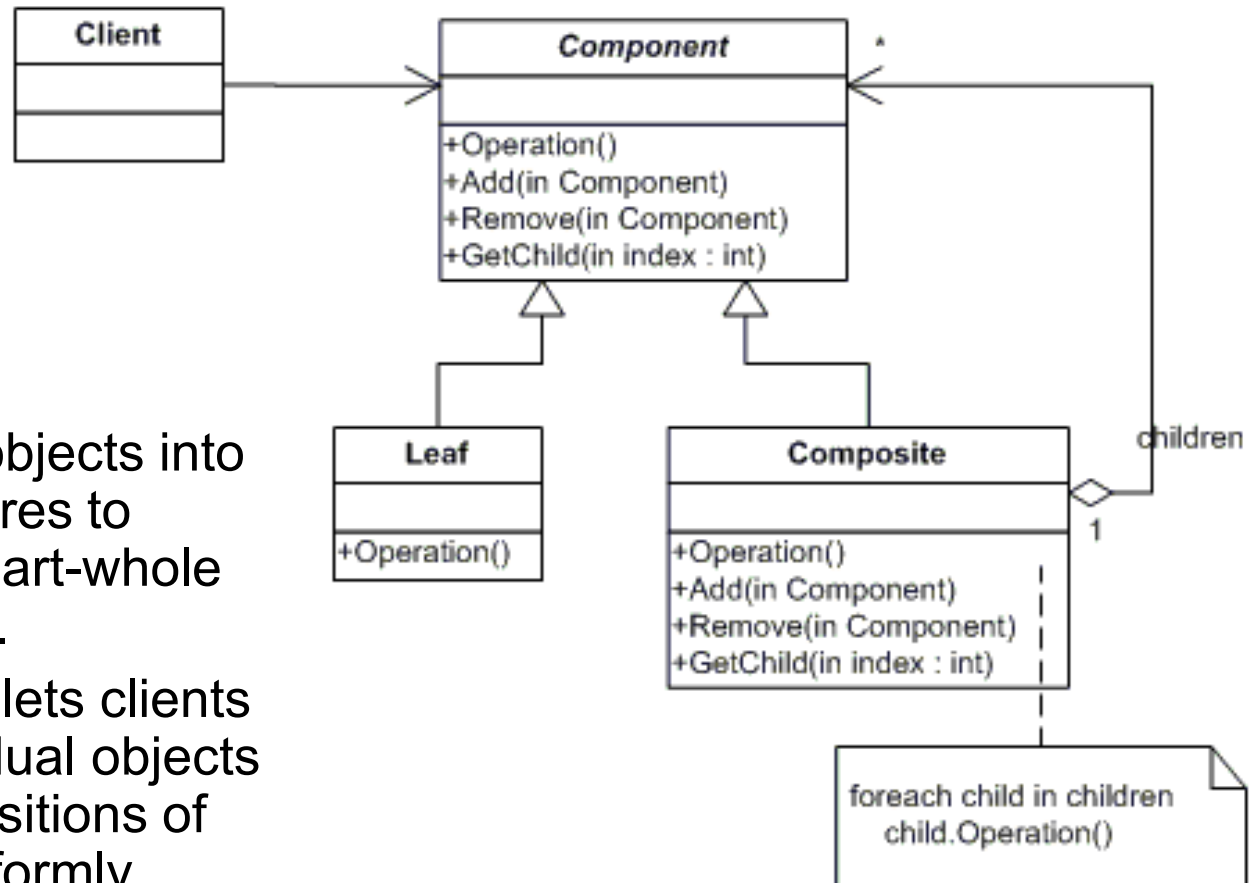


Problem 4

Composite

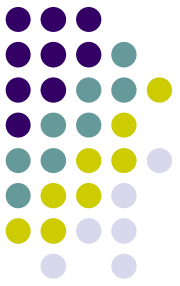
- **Composite**

- Compose objects into tree structures to represent part-whole hierarchies.
- Composite lets clients treat individual objects and compositions of objects uniformly



Problem 5

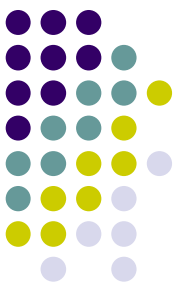
Accessing



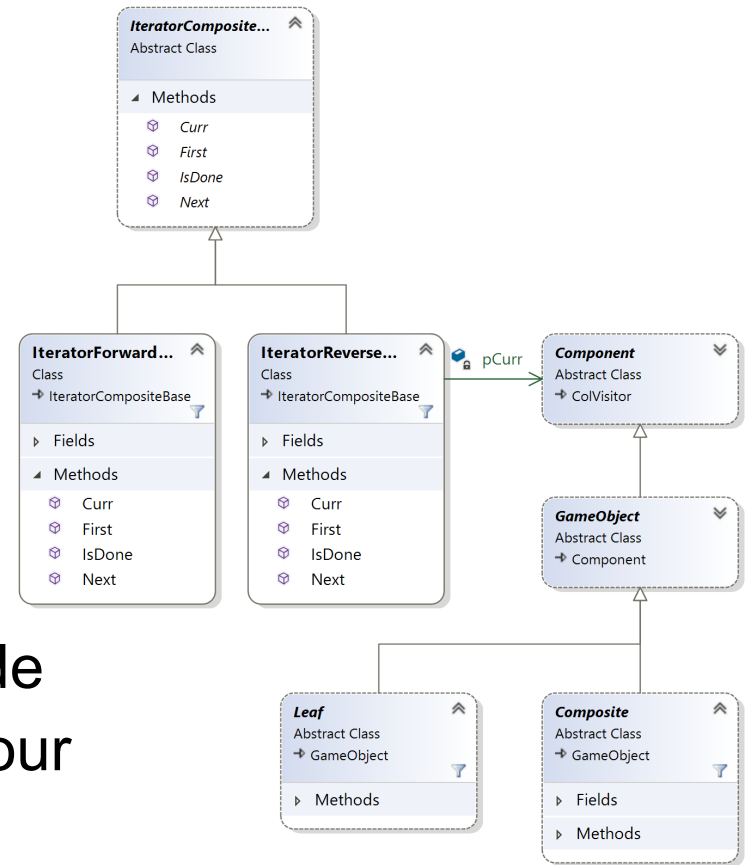
- Data is in a container
 - Link List
 - Tree
 - Composite
- How do I use the data?
 - Able to change the holding container freely
 - No direct access links
 - No knowledge of the internals of the container

Problem 5

Getting data

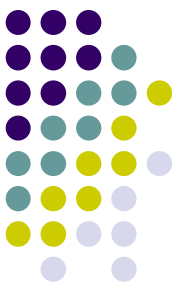


- **Iterators** to the rescue
 - You don't use them
 - Its so easy to use the data directly
 - Why Bother?
- **Serious Benefits**
 - Allows you to control access
 - Allows you to restructure code
 - Encapsulate complexity of your storage mechanism
 - Link Lists or Trees (doesn't matter)



Problem 6

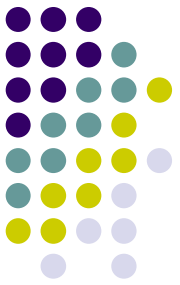
Triggering



- How do you trigger
 - an action?
 - a group of actions?
- Delayed or timed group of actions?
 - Use the Command Pattern with a priority queue for timed events.
 - You can add / remove events relative to an absolute time

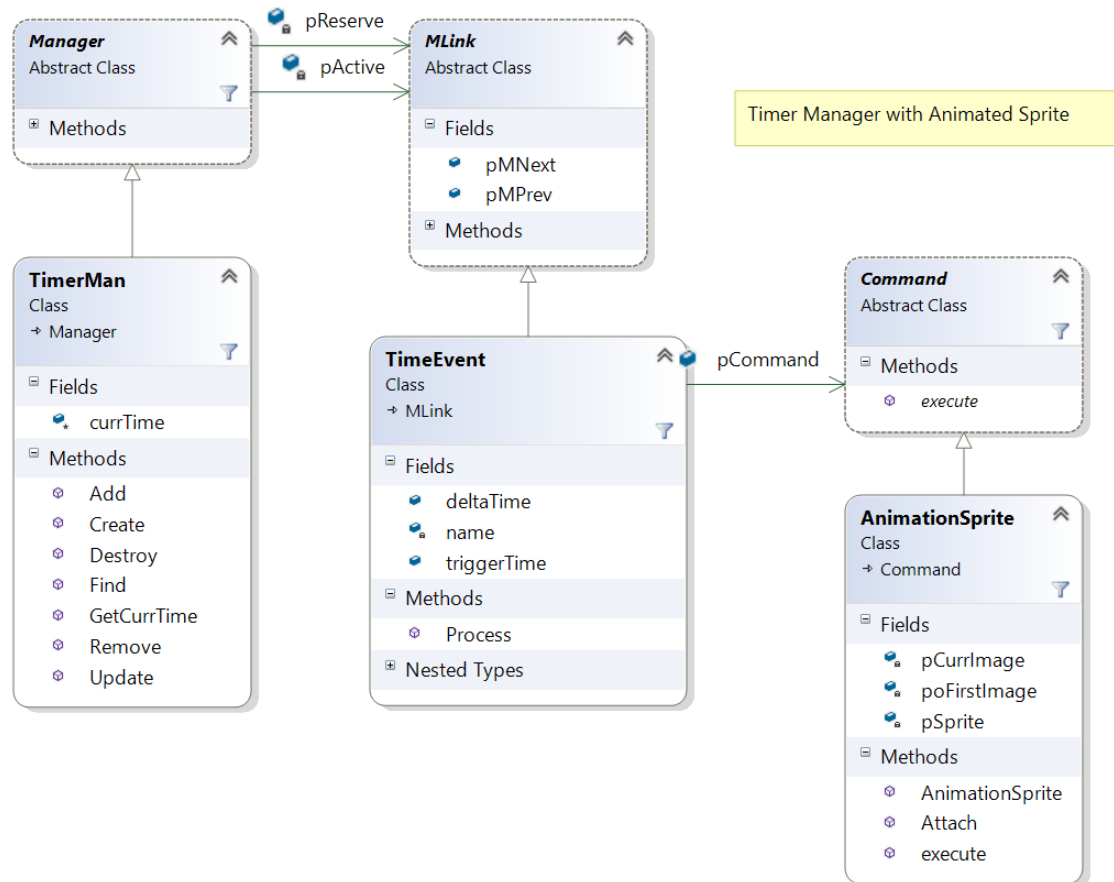
Problem 6

Timer Events



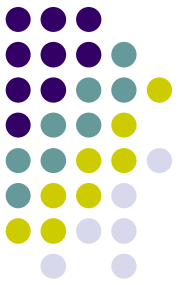
- **Command Patten**

- Encapsulate a request in an object
- Allows the parameterization of clients with different requests
- Allows saving the requests in a queue



Problem 7

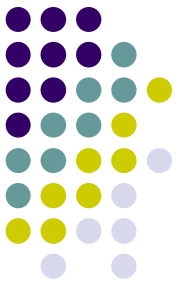
Notification



- How do you notify...
 - Aliens hit the wall?
 - Change directions
 - UFO was shot?
 - Start dead sequence
- Goal
 - Want systems to handle there stuff
 - Delegation and encapsulated behavior

Problem 7

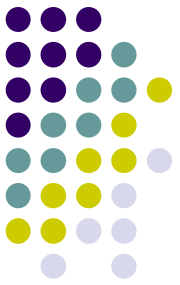
Notification



- Where do you **NOT** do this...
 - No knowledge in Collision system
 - Not in Visitor Pattern
 - Not polling by Game System
 - “Did you collide?”
- Use the Observer pattern
 - Event happen notified everyone who “cares” about that event

Problem 7

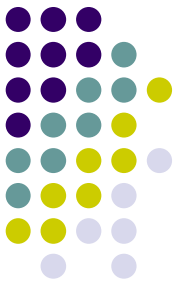
Notification - Example



- Missile hits UFO
 - Notify the Player's Ship
 - Only one missile can be moving at a time
 - Notify the Score
 - Add points
 - Notify animation death sequence
 - Have the UFO die
 - Changing Sprites over time
 - Triggering sound effects

Problem 7

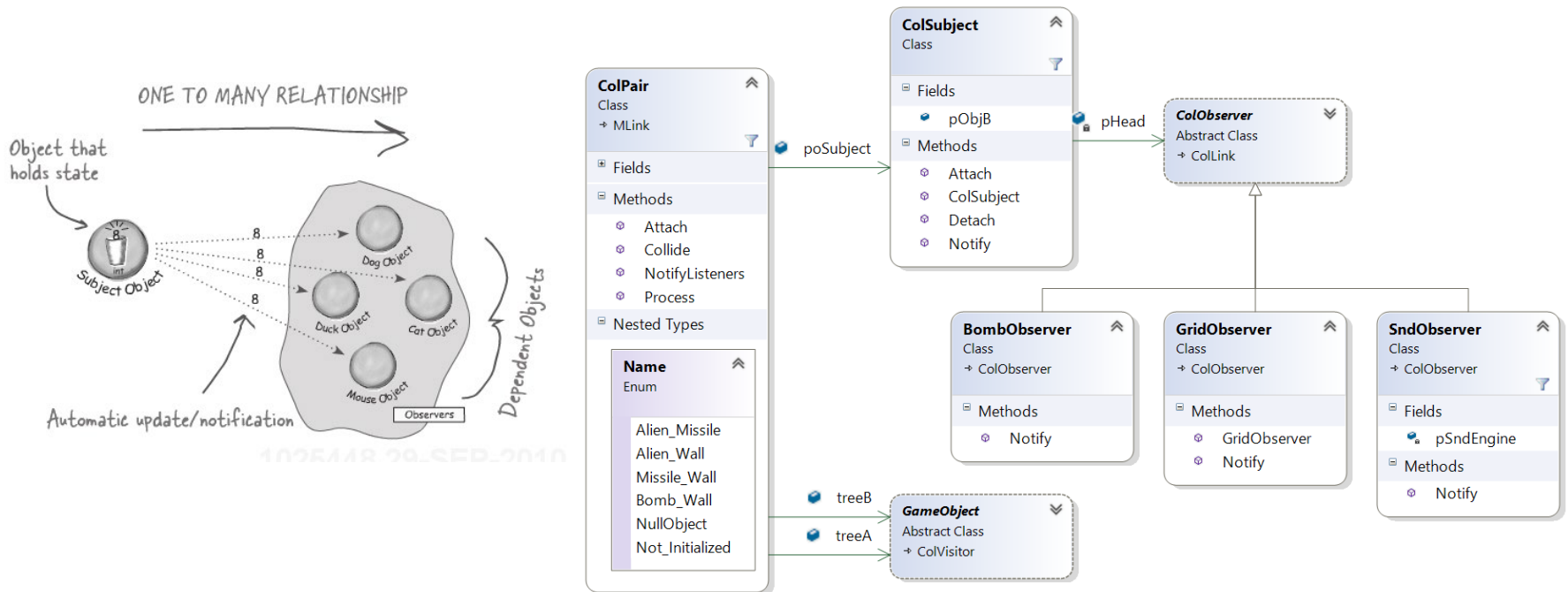
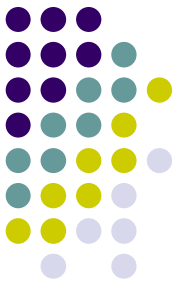
Notification –Triggering



- Inside the Visitor of appropriate collision
 - Call all the Listeners associated to the collision
 - May be one or several listeners
 - You don't care (data driven)
- ***REPEAT***
 - No work should be done in the Visitor
 - Only Delegation
 - Kick the can baby!

Problem 7

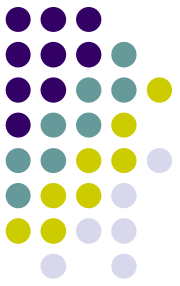
Notification



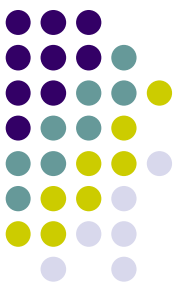
- Magic secret sauce
 - Systems communicated when needed by **Observers**

Problem 8

Collisions



- Big problem
 - Very quick efficient way to discriminate collision decisions?
 - Who collided with whom?
- Requirements
 - No conditionals
 - No switch statements
 - Extensible
 - Scalable



Problem 8

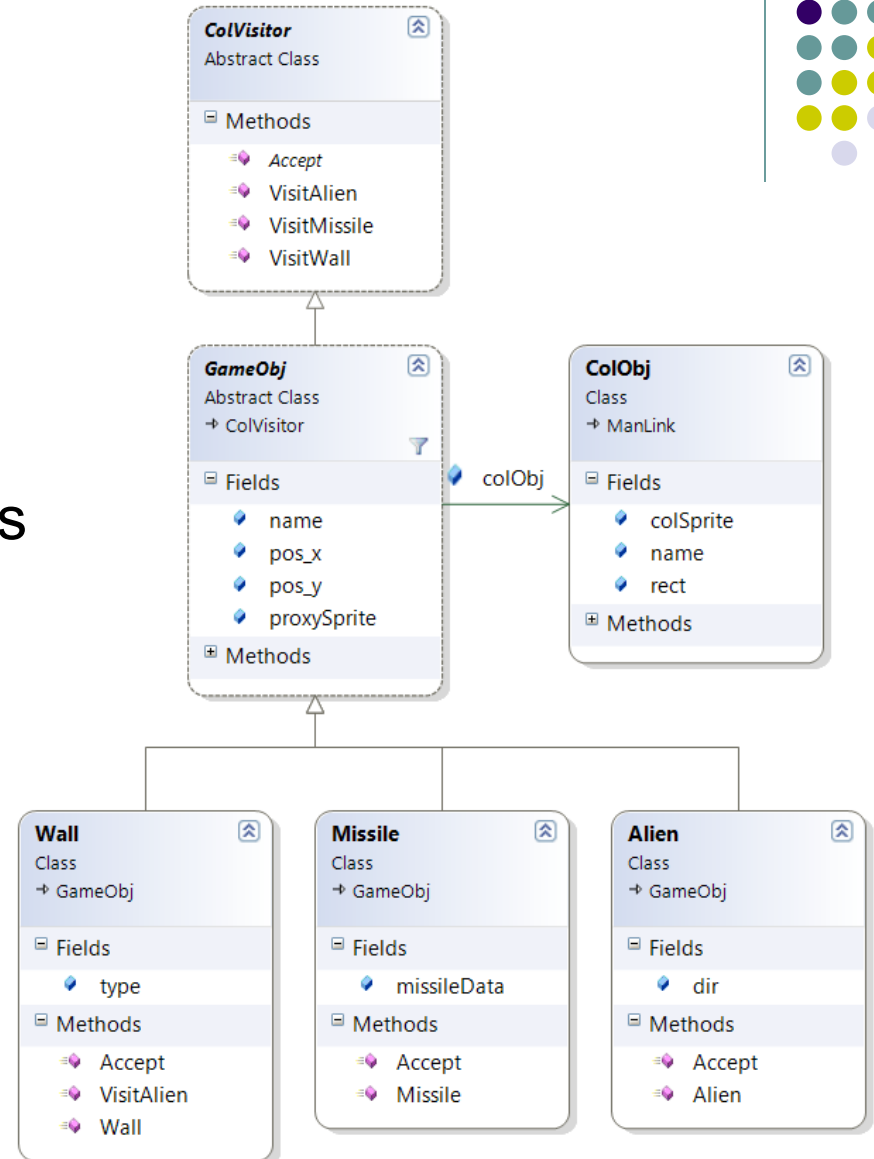
Who collided?

- Visitor pattern
 - This pattern is really being used backwards
 - Only the double dispatch portion
- Each GameObject
 - Has **Accept()** method
 - Has sparsely populated **Visit()** methods for each accepted collision type.
- My Opinion
 - Clean solution to a sticky problem

Problem 8

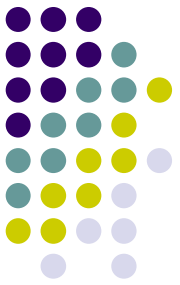
Who collided?

- Pitfall
 - Colliding at a too low-level
 - Need to collide at GameObj level
- Only have Concrete Objects
 - Specialized GameObjs
 - Wall
 - Missile
 - Alien
- **Visitor Pattern**
 - Best for resolving which object collides



Problem 9

Special Behaviors



- How do I specialize my behavior?
 - Have the system automatically behave differently
- Bombs dropping
 - 3 different Modes
 - The use different Sprites
 - Different drawing behaviors as they fall

Bomb.A



Bomb.B

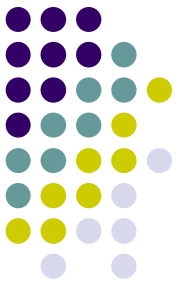


Bomb.C



Problem 9

Special Behaviors

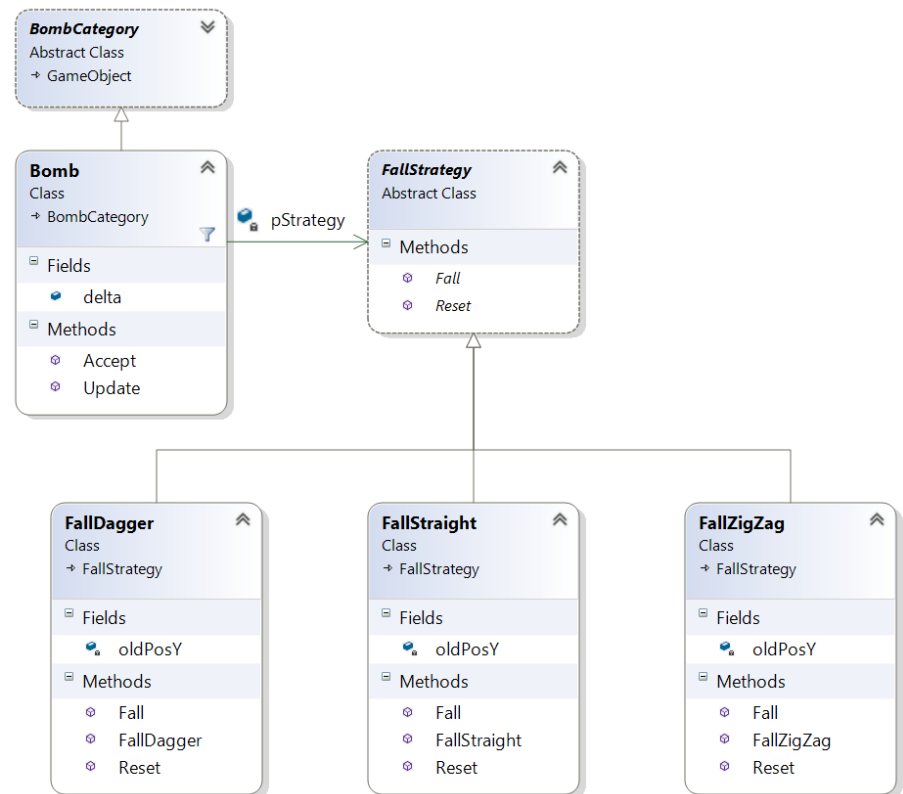


- **Strategy Pattern**

- Easy way to specialize behaviors
- Write the behavior in isolation from the others

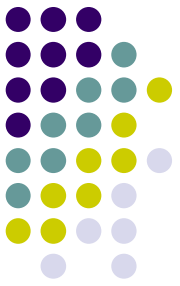
- **Organize**

- Remove Special cases
- Easy to extend



Problem 10

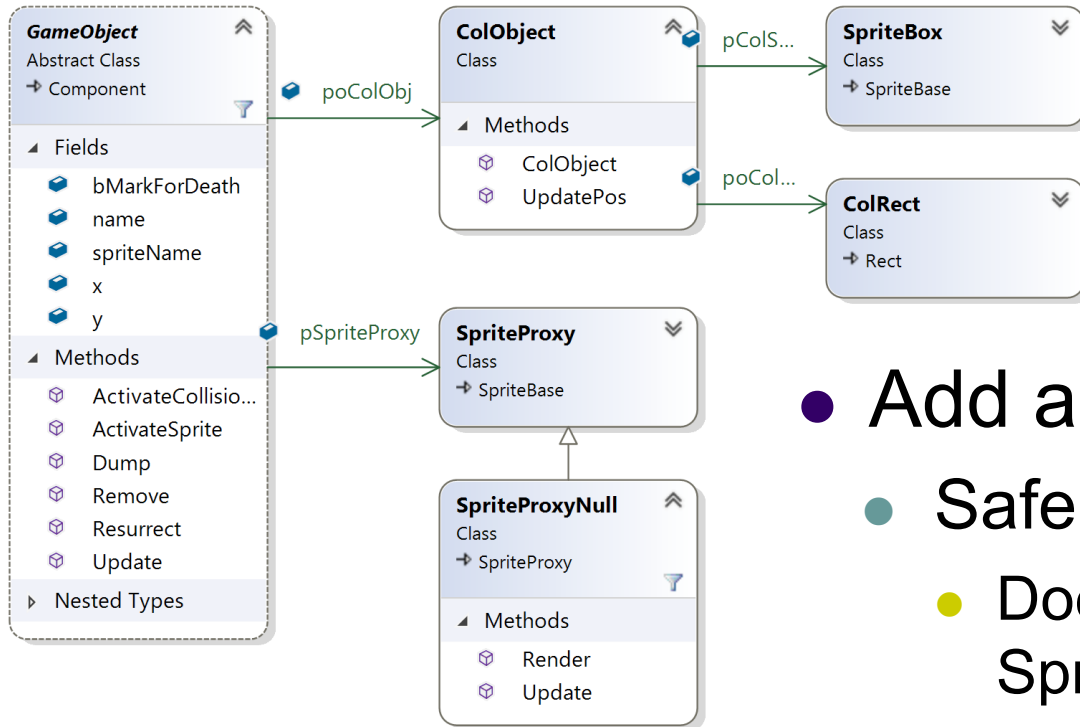
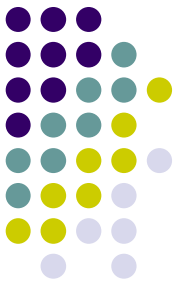
Special cases



- How do you deal with conditionals?
 - Do the same work each and every time
- Sometimes the object doesn't have any real work to complete
 - So do nothing...
 - no edge cases
- Null Object to the rescue!

Problem 10

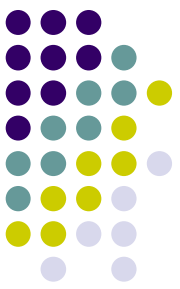
Null Object



- Add a ***Null Object***
 - Safe object
 - Does nothing by the Sprite System
 - No Special cases in code
 - Good!

Problem 11

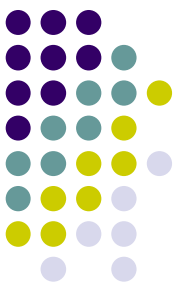
Game Modes



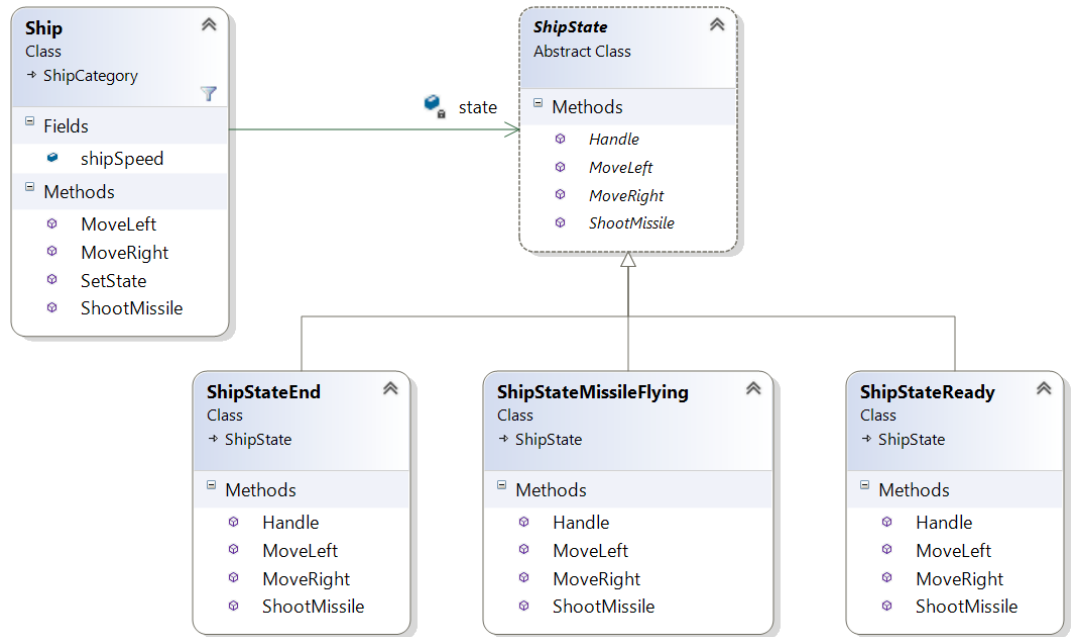
- How do I cleanly handle different modes in the game?
 - Control the game flow
 - Attract → Intro → Game → GameOver → Attract
 - Control the modes of game objects
 - Only one missile can fly at a time
 - Missile Ready
 - Missile Flying
 - Alien life
 - Moving (Alive - collidable)
 - Death Sequence (not collidable)

Problem 11

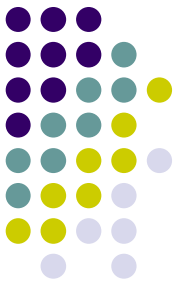
Modes



- Use the **State Pattern**
 - Allows an object to alter its behavior when its internal state changes.
 - Unlike a procedural state machine, the State Pattern represents state as a full-blown class



Thank You!



- Questions?

